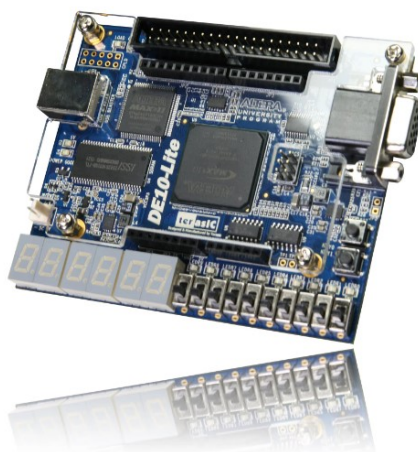


Licenciatura em Engenharia Informática e de Computadores



RELATÓRIO DO 3^o TRABALHO

Soma de quadrados de números naturais

Trabalho realizado por:

Dinis Melo nº 51500

Francisco Mota nº 52752

Turma: 11D

Docente: Pedro Miguens

Lógica e Sistemas Digitais

2024 / 2025 inverno

Índice

1	Objetivo	3
2	Descrição do Circuito a Desenvolver	3
3	Desenvolvimento do Projeto	4
3.1	Simplificação e Expressões Lógicas	5
3.2	Diagrama de Blocos	8
4	Resultados e Discussão	9
4.1	Simulação	9
4.2	Testes	10
5	Conclusão	10
6	Referências	10
7	VHDL	11
7.1	HA.vhd	11
7.2	FA.vhd	11
7.3	adder4bit.vhd	12
7.4	adder8bit.vhd	13
7.5	X2.vhd	15
7.6	FFD.vhd	16
7.7	datapath.vhd	17
7.8	ASM.vhd	19
7.9	reg1bit.vhd	21
7.10	reg4bit.vhd	21
7.11	reg8bit.vhd	23
7.12	MUX.vhd	24
7.13	MUX8bit.vhd	24
7.14	decoderHex.vhd	26
7.15	bin2dec.vhd	27
7.16	TLAB3.vhd	28

1 Objetivo

O objetivo deste trabalho é desenvolver um circuito digital que seja capaz de calcular o somatório de quadrados de numeros naturais fornecidos pelo usuário com base em VHDL estrutural, simular e implementar o circuito na placa de desenvolvimento DE10-Lite da intel.

2 Descrição do Circuito a Desenvolver

Pretende-se implementar um circuito digital que calcule o somatório dos quadrados de múltiplas entradas de 4 bits fornecidas pelo usuário, e que enquanto a entrada *Step* permanecer ativa seja capaz de apresentar o resultado intermédio de xi^2 nos *displays*

As entradas serão as seguintes, X o numero de 4 bits fornecido pelo utilizador, MCLK, o sinal de *clock* principal que sincroniza todas as operações do circuito, Rst, o sinal de *reset* que reinicia o sistema, ou seja, tanto sumatorio como o *carry* in voltarão a 0, *Step*, o sinal que controla a alternância entre a saída de X^2 e o somatório acumulado e finalmente *Start* ,o sinal que inicia a sequência de cálculo do circuito. O resultado do cálculo é exibido atraves da saída *Sum*, que pode ser o quadrado da entrada X ou o somatório acumulado até ao momento, dependendo, como já mencionado, do valor de *Step*, que quando ativado, exhibe o resultado intermédio de xi^2 e quando desativado, apresenta o sumatório acumulado. O sinal Cy indica a presença de *carry* gerado nas operações de soma do sumatorio. Para além disso, o valor calculado é exibido em três displays de 7 segmentos, HEX0, HEX e HEX2, que mostram o valor de *Sum* em três digitos. Dessa forma, as saídas fornecem tanto o resultado de X^2 quanto o somatório calculado, com o *carry* e a exibição visual dos resultados em tempo real.

3 Desenvolvimento do Projeto

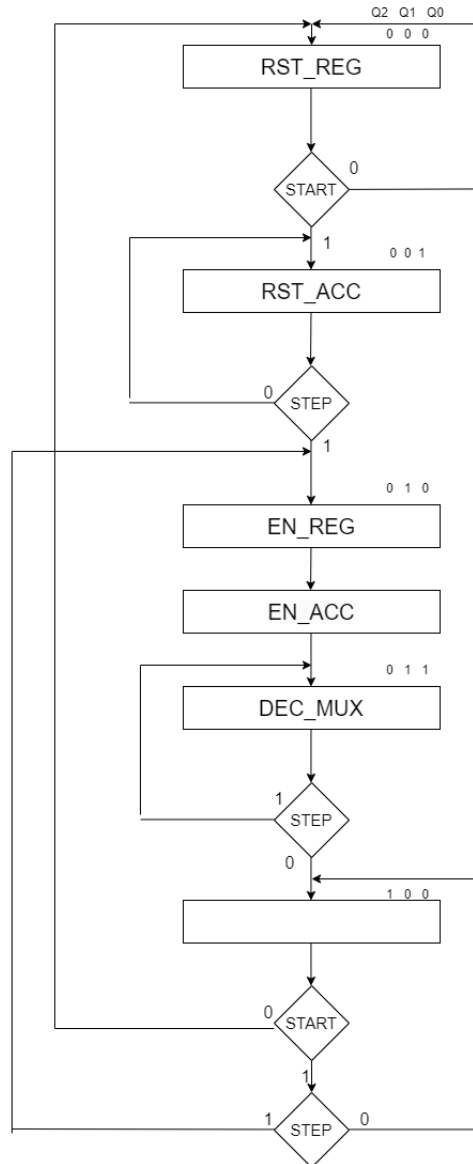
Para realizar este projeto, começamos pela realização do diagrama de blocos do projeto, identificando os principais módulos e suas funções, como o módulo de cálculo de X^2 , os registros de 4 e 8 bits e o decodificador para os displays de 7 segmentos. Além disso, determinamos as expressões lógicas para a interação entre esses módulos, a partir do *ASM Chart* também realizado pelo grupo.

Seguindo esta análise, descrevemos o circuito em VHDL, respeitando rigorosamente a hierarquia e a funcionalidade definida pelo diagrama de blocos. Depois de implementar a descrição do circuito, realizamos simulações detalhadas para validar o comportamento esperado do sistema, verificando se o quadrado de X era calculado corretamente, se o somatório acumulado estava sendo atualizado conforme as entradas e se a alternância entre as saídas estava a funcionar conforme esperado. Também validamos o comportamento do sinal de *Carry* (Cy), que indica se a soma dos quadrados está fora do domínio $[0, +255]$.

Após as simulações, fizemos a atribuição de pinos e aplicamos o circuito descrito na placa DE10-Lite e realizamos testes experimentais para validar o funcionamento prático do sistema. Durante os testes, verificamos que as operações do cálculo de X^2 , o somatório acumulado e alternância de saída estavam a ser executadas corretamente. Além disso, confirmamos que os displays de 7 segmentos exibiam o resultado de forma adequada e que o sinal de *Carry* estava a ser ativado corretamente quando necessário.

3.1 Simplificação e Expressões Lógicas

Para as expressões lógicas dos estados e dos *Flip Flops* Para chegar às expressões lógicas dos estados e dos *Flip Flops*, fizemos primeiro o *ASM Chart* que se segue:



A partir de este fluxograma e recorrendo a mapas de *Karnaugh* chegámos às expressões lógicas dos *Flip FLlops*

$$D0 = (\overline{Q_2} \cdot \overline{Q_0} \cdot \overline{Q_1} \cdot \text{start}) + (Q_0 \cdot \overline{Q_1} \cdot \overline{Q_2} \cdot \text{step}) + (Q_0 \cdot \overline{Q_2} \cdot Q_1 \cdot \text{step}) + (\overline{Q_0} \cdot Q_1 \cdot \overline{Q_2})$$

$$(F0 = \text{Start}) (F1 = \overline{\text{Step}}) (F2 = \text{Step})$$

$$D1 = (\overline{Q_0} \cdot \overline{Q_1} \cdot Q_2 \cdot \text{start} \cdot \text{step}) + (\overline{Q_2} \cdot Q_0 \cdot \text{step}) + (\overline{Q_0} \cdot Q_1 \cdot \overline{Q_2})$$

(F0 = Step) (F1 = start · step)

$$D2 = (\overline{Q_0} \cdot \overline{Q_1} \cdot Q_2 \cdot \text{start} \cdot \overline{\text{step}}) + (Q_0 \cdot Q_1 \cdot \overline{Q_2} \cdot \overline{\text{step}}) \\ (F1 = \text{start} \cdot \overline{\text{step}})(F0 = \overline{\text{step}})$$

Após termos as expressões lógicas dos *Flip Flops* registámos também as expressões lógicas de cada estado:

$$ENReg = \overline{Q(2)} \cdot Q1 \cdot \overline{Q(0)};$$

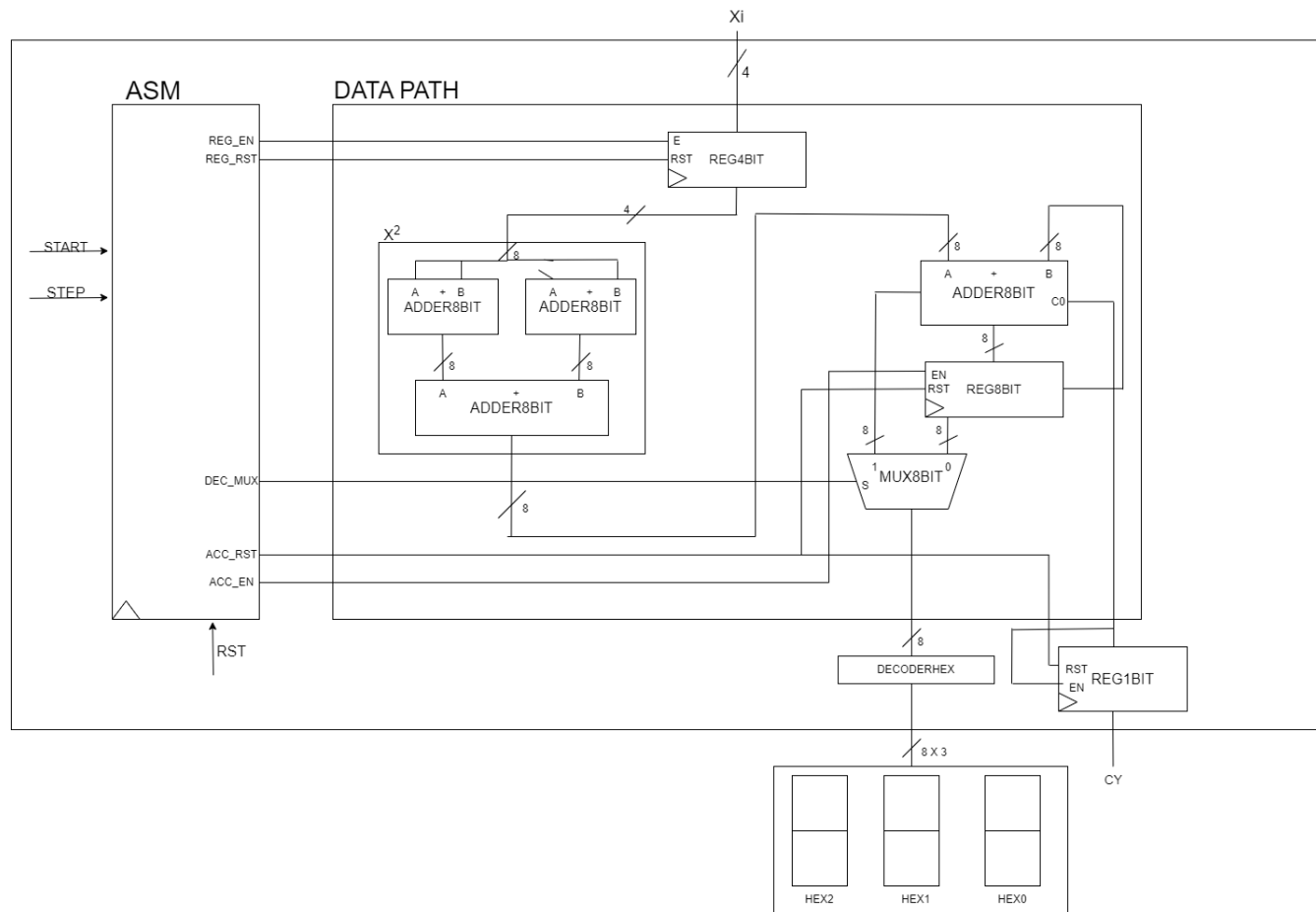
$$RSTReg = \overline{Q(2)} \cdot \overline{Q(1)} \cdot \overline{Q(0)};$$

$$DECMux = \overline{Q(2)} \cdot Q1 \cdot Q0;$$

$$ENACC = \overline{Q(2)} \cdot Q1 \cdot \overline{Q(0)};$$

$$RSTACC = \overline{Q(2)} \cdot \overline{Q(1)} \cdot Q0;$$

3.2 Diagrama de Blocos



Este foi o diagrama de blocos a que chegámos, nele podemos ver como funciona o projeto.

Data Path:

X^2 : Utiliza três blocos de somadores de 8 bits, que fazem a soma de ands de bits, de forma a que seja possível fazer a multiplicação do X por ele mesmo.

Somador de 8 bits: Feito por dois somadores de 4 bits, é usado duas vezes neste projeto, uma delas como referido, no X^2 e a outra para fazer a soma entre o X^2 e o resultado da soma anterior.

MUX: Escolhe se aparece o X^2 no display de 7 segmentos ou se aparece a saída Sum, dependendo de Step.

Reg4bit: Guarda o valor de Xi em 4 bits. Reg8bit: Guarda o valor do somador de 8 bits, para que este valor seja utilizado novamente na próxima soma.

Reg1bit: Guarda o valor de CY de forma a que, a partir do momento que CY está ativo, este fique sempre ativo, até haver reset.

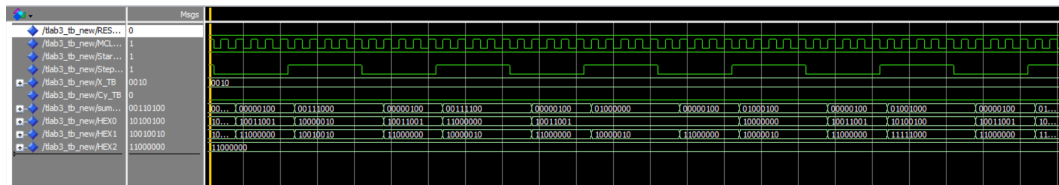
DecoderHex: Este bloco converte o resultado de 8 bits para um número decimal, este resultado é mostrado no display de 7-segmentos e para os LEDs da placa.

ASM: Este bloco é uma máquina de estados, ou seja, um módulo de controlo. O ASM recebe sinais externos ao circuito, sinais provenientes do Data Patch, para conseguir determinar que ações vai fazer sobre o mesmo. Este utiliza 3 Flip Flops para poder realizar as mudanças de estados necessárias para o circuito funcionar bem.

4 Resultados e Discussão

4.1 Simulação

Depois de termos feito a implementação em VHDL do projeto, simulámos no ModelSim para ver se os resultados teóricos eram iguais ao que tínhamos feito nas tabelas de verdade, o que se verificou:



4.2 Testes

A simulação no ModelSim permitiu verificar o comportamento correto do circuito antes da sua implementação física, facilitando a identificação de possíveis erros. Após a simulação e implementação do circuito na DE10-Lite, os resultados experimentais confirmaram o funcionamento correto do circuito. As saídas dos LEDs correspondiam às expectativas teóricas e correspondiam também ao resultado do *display* de 7 segmentos para todas as combinações de entradas testadas.

5 Conclusão

O objetivo estipulado foi alcançado com sucesso. Este terceiro trabalho permitiu-nos aprofundar conhecimentos teóricos e práticos em diversas áreas como, por exemplo, chegar a um diagrama de blocos a partir do problema dado, chegar a um *ASM Chart* e às respectivas expressões lógicas do mesmo. Assim, todos os aspetos referidos foram meios que futuramente irão se tornar uma base para a aplicação de todos estes conhecimentos teórico-práticos nos futuros projetos do curso.

6 Referências

- (1) Pedro Miguens Matutino, Sérgio André Guia de Instalação do Quartus Prime Lite Edition 20.1.1.720, 2022, ISEL.
- (2) Terasic DE10-lite User Manual, 2020, Terasic.

7 VHDL

7.1 HA.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity HA is
port (
    A : in std_logic;
    B : in std_logic;
    S : out std_logic;
    C0 : out std_logic
);
end HA;

architecture structural of HA is
begin
    S <= A xor B;
    C0 <= A and B;
end structural;
```

7.2 FA.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity FA is
port (
    A : in std_logic;
    B : in std_logic;
    Ci : in std_logic;
    S : out std_logic;
    C0 : out std_logic
);
end FA;
```

```

architecture structural of FA is
  component HA is
  port (
    A : in std_logic;
    B : in std_logic;
        S : out std_logic;
    C0 : out std_logic
  );
end component;

signal addAB : std_logic;
signal carry : std_logic_vector(1 downto 0);

begin

  U1 : HA port map ( A => A, B => B, S => addAB, C0 => carry(0));
  U2 : HA port map ( A => addAB, B => Ci, S => S, C0 => carry(1));

  C0 <= carry(0) or carry(1);

end structural;

```

7.3 adder4bit.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity adder4bit is
  port (
    A : in std_logic_vector(3 downto 0);
    B : in std_logic_vector(3 downto 0);
    Ci : in std_logic;
    S : out std_logic_vector(3 downto 0);
    C0 : out std_logic
  );

```

```

);
end adder4bit;

architecture structural of adder4bit is
component FA is
port (
    A  : in std_logic;
    B  : in std_logic;
    Ci : in std_logic;
    S  : out std_logic;
    C0 : out std_logic
);
end component;
signal carry : std_logic_vector(3 downto 1);

begin

U1 : FA port map ( A => A(0), B => B(0), Ci => Ci, S => S(0), C0 => carry(1));
U2 : FA port map ( A => A(1), B => B(1), Ci => carry(1), S => S(1), C0 => carry(2));
U3 : FA port map ( A => A(2), B => B(2), Ci => carry(2), S => S(2), C0 => carry(3));
U4 : FA port map ( A => A(3), B => B(3), Ci => carry(3), S => S(3), C0 => C0);

end structural;

```

7.4 adder8bit.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity adder8bit is
port(

    A  : in std_logic_vector(7 downto 0);
    B  : in std_logic_vector(7 downto 0);
    Ci : in std_logic;

```

```

        S : out std_logic_vector(7 downto 0);
        C0 : out std_logic

    );
end adder8bit;

architecture structural of adder8bit is
    component adder4bit is
    port (
        A : in std_logic_vector(3 downto 0);
        B : in std_logic_vector(3 downto 0);
        Ci : in std_logic;
        S : out std_logic_vector(3 downto 0);
        C0 : out std_logic
    );
end component;

    signal carry,C0_S: std_logic;
    signal A_signal1,A_signal2,B_signal2,B_signal1,S_signal1,S_signal2 : std_logic_vector(3 downto 0)

begin
    A_signal1 <= (A(3) & A(2) & A(1) & A(0));
    A_signal2 <= (A(7) & A(6) & A(5) & A(4));
    B_signal1 <= (B(3) & B(2) & B(1) & B(0));
    B_signal2 <= (B(7) & B(6) & B(5) & B(4));
    U1 : adder4bit port map (A=>A_signal1,B=>B_signal1,S=>S_signal1,Ci=>'0',C0=>carry);
    U2 : adder4bit port map (A=>A_signal2,B=>B_signal2,S=>S_signal2,Ci=>carry,C0=>C0_S);
    S<= (S_signal2(3) & S_signal2(2) & S_signal2(1) & S_signal2(0) & S_signal1(3) & S_signal1(2) & S_
    C0 <= C0_S;
end structural;

```

7.5 X2.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity X2 is
port(
    X:in std_logic_vector(3 downto 0);
    X2X:out std_logic_vector(7 downto 0)
);
end X2;

architecture structural of X2 is
component adder8bit is
port(
    A : in std_logic_vector(7 downto 0);
    B : in std_logic_vector(7 downto 0);
    Ci : in std_logic;
    S : out std_logic_vector(7 downto 0);
    CO : out std_logic
);
end component;

signal X_Signal,XX0,XX1,XX2,XX3,S_Signal1,S_Signal2,S_Signal3: std_logic_vector(7 downto 0);
signal X1 , X2 , X3, X0 : std_logic;
begin
X_Signal<= ('0'&'0'&'0'&'0'&X(3)&X(2)&X(1)&X(0));
X1<= X(1);
X2<= X(2);
X3<= X(3);
X0<= X(0);
XX0<=('0' & '0' & '0' & '0' & (X3 and X0) & (X2 and X0) & (X1 and X0) & X0 );
XX1<=('0'&'0'&'0'&(X3 and X1) & (X2 and X1) & X1 & (X0 and X1) & '0');
XX2<=('0'&'0'&(X3 and X2) & X2 & (X1 and X2) & (X0 and X2) & '0' & '0');
XX3<=('0'& X3 & (X2 and X3) & (X1 and X3) & (X0 and X3) & '0' & '0' & '0');

U1 : adder8bit port map (A=>XX0,B=>XX1,Ci=>'0',S=>S_Signal1 );
U2 : adder8bit port map (A=>XX2,B=>XX3,Ci=>'0',S=>S_Signal2 );
```

```

U3 : adder8bit port map (A=>S_Signal1,B=>S_Signal2,Ci=>'0',S=>S_Signal3 );

X2X <= S_Signal3;
end structural;

```

7.6 FFD.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY FFD IS
PORT(
    CLK : in std_logic;
    RESET : in STD_LOGIC;
    SET : in std_logic;
    D : IN STD_LOGIC;
    EN : IN STD_LOGIC;
    Q : out std_logic
);
END FFD;

ARCHITECTURE logicFunction OF FFD IS

BEGIN

Q <= '0' when RESET = '1' else '1' when SET = '1' else D WHEN rising_edge(clk) and EN = '1';

END LogicFunction;

```

7.7 datapath.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity datapath is
    port(
        xi: in std_logic_vector(3 downto 0);
        clk: in std_logic;
        EN_Reg,RST_Reg,Dec_Mux,EN_ACC,RST_ACC : in std_logic;

        R : out std_logic_vector(7 downto 0);
        CY : out std_logic
    );

end datapath;

architecture structural of datapath is

    component mux8bit is
        port
        (
            A : in std_logic_vector (7 downto 0);
            B : in std_logic_vector (7 downto 0);
            Dec_Mux: in std_logic;
            Y : out std_logic_vector (7 downto 0)
        );
    end component;

    component X2 is
        port(

            X:in std_logic_vector(3 downto 0);

            X2X:out std_logic_vector(7 downto 0)

        );
    end component;
```

```

component reg4bit is
port(
x : in std_logic_vector(3 downto 0);
clk : in std_logic;
RST : in std_logic;
EN : in std_logic;
xout : out std_logic_vector(3 downto 0)
);
end component;

component adder8bit is
port(

    A : in std_logic_vector(7 downto 0);
    B : in std_logic_vector(7 downto 0);
    Ci : in std_logic;
    S : out std_logic_vector(7 downto 0);
    C0 : out std_logic

);
end component;

component reg8bit is
port(
x8 : in std_logic_vector(7 downto 0);
clk : in std_logic;
RST : in std_logic;
EN : in std_logic;
xout8 : out std_logic_vector(7 downto 0)
);
end component;

signal xout4 : std_logic_vector(3 downto 0);
signal X2X_Signal,R_Signal,S_adder8bit,REG8_OUT : std_logic_vector(7 downto 0);
signal CY_SIGNAL: std_logic;
begin

```

```

U1 : reg4bit port map(x=>xi,clk=>clk,RST=>RST_Reg,EN=>EN_Reg,xout=>xout4);

U2 : X2 port map(X=>xout4,X2X=>X2X_Signal);

U3 : adder8bit port map(A=>X2X_Signal,B=>REG8_OUT,Ci=>'0',CO=>CY_SIGNAL,S=>S_adder8bit);

U4 : reg8bit port map(x8=>S_adder8bit,clk=>clk,EN=>EN_ACC,RST=>RST_ACC,xout8=>REG8_OUT);

U5 : mux8bit port map(A=>S_adder8bit,B=>X2X_Signal,Dec_Mux=>Dec_Mux,Y=>R_Signal);

CY <= CY_SIGNAL;
R <= R_Signal;

end structural;

```

7.8 ASM.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity ASM is
port(
    reset      : in std_logic;
    clk        : in std_logic;
    start      : in std_logic;
    step       : in std_logic;
    EN_Reg,RST_Reg,Dec_Mux,EN_ACC,RST_ACC: out std_logic
);
end ASM;

architecture structural of ASM is

    component FFD IS
    PORT( CLK : in std_logic;

```

```

        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
    );
END component;

signal D2, D1, D0, Q2, Q1, Q0 : std_logic;

begin

    -- Flip-flop's
    Flip_Flop_Q2: FFD port map( CLK => clk, RESET => reset, SET => '0', D => D2, EN => '1', Q => Q2);
    Flip_Flop_Q1: FFD port map( CLK => clk, RESET => reset, SET => '0', D => D1, EN => '1', Q => Q1);
    Flip_Flop_Q0: FFD port map( CLK => clk, RESET => reset, SET => '0', D => D0, EN => '1', Q => Q0);

    -- Generate Next State
    D2 <= ((not(Q0) and not(Q1) and Q2 and start and not(step)) or (Q0 and Q1 and not(Q2) and not(step)))
    D1 <= ((not(Q0) and not(Q1) and Q2 and start and step) or (not(Q2) and Q0 and step) or (not(Q0) and Q1 and not(Q2) and not(step)))
    D0 <= ((not(Q2) and not(Q1) and not(Q0) and start) or (Q0 and not(Q1) and not(Q2) and not(step)))

    -- Generate outputs
    EN_Reg<=not(Q2) and Q1 and not(Q0);
    RST_Reg<=not(Q2) and not(Q1) and not(Q0);
    Dec_Mux<=not(Q2) and Q1 and Q0;
    EN_ACC<=not(Q2) and Q1 and not(Q0);
    RST_ACC<=not(Q2) and not(Q1) and Q0;

end structural;

```

7.9 reg1bit.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity reg1bit is
port(
  x : in std_logic;
  clk : in std_logic;
  RST : in std_logic;
  EN : in std_logic;
  xout : out std_logic
);
end reg1bit;

architecture structural of reg1bit is
  component FFD is
  PORT(
    CLK : in std_logic;
    RESET : in STD_LOGIC;
    SET : in std_logic;
    D : IN STD_LOGIC;
    EN : IN STD_LOGIC;
    Q : out std_logic
  );
END component;

begin
  T0: FFD port map(CLK => clk, RESET => RST, SET => '0', D => x, EN => EN, Q => xout);

end structural;
```

7.10 reg4bit.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
```

```

entity reg4bit is
port(
x : in std_logic_vector(3 downto 0);
clk : in std_logic;
RST : in std_logic;
EN : in std_logic;
xout : out std_logic_vector(3 downto 0)
);
end reg4bit;

architecture structural of reg4bit is
component FFD is
PORT(
        CLK : in std_logic;
        RESET : in STD_LOGIC;
        SET : in std_logic;
        D : IN STD_LOGIC;
        EN : IN STD_LOGIC;
        Q : out std_logic
    );
END component;

begin
T0: FFD port map(CLK => clk, RESET => RST, SET => '0', D => x(0), EN => EN, Q => xout(0));

T1: FFD port map(CLK => clk, RESET => RST, SET => '0', D => x(1), EN => EN, Q => xout(1));

T2: FFD port map(CLK => clk, RESET => RST, SET => '0', D => x(2), EN => EN, Q => xout(2));

T3: FFD port map(CLK => clk, RESET => RST, SET => '0', D => x(3), EN => EN, Q => xout(3));

end structural;

```

7.11 reg8bit.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity reg8bit is
port(
x8 : in std_logic_vector(7 downto 0);
clk : in std_logic;
RST : in std_logic;
EN : in std_logic;
xout8 : out std_logic_vector(7 downto 0)
);
end reg8bit;

architecture structural of reg8bit is
component reg4bit is
port(
x : in std_logic_vector(3 downto 0);
clk : in std_logic;
RST : in std_logic;
EN : in std_logic;
xout : out std_logic_vector(3 downto 0)
);
end component;

signal X_Signal1,X_Signal2,Xout_Signal1,Xout_Signal2 : std_logic_vector(3 downto 0);
begin
X_Signal1<= (x8(3) & x8(2) & x8(1) & x8(0));
X_Signal2<= (x8(7) & x8(6) & x8(5) & x8(4));
U1 : reg4bit port map(x=>X_Signal1,clk=>clk,RST=>RST,EN=>EN,xout => Xout_Signal1);
U2 : reg4bit port map(x=>X_Signal2,clk=>clk,RST=>RST,EN=>EN,xout => Xout_Signal2);
xout8<=(Xout_Signal2(3) & Xout_Signal2(2) & Xout_Signal2(1) & Xout_Signal2(0) & Xout_Signal1(3) &
Xout_Signal1(2) & Xout_Signal1(1) & Xout_Signal1(0));
end structural;
```

7.12 MUX.vhd

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity MUX is
port(
    A, B : in STD_LOGIC_VECTOR(3 downto 0);
    Dec_Mux : in STD_LOGIC;
    Y : out STD_LOGIC_VECTOR(3 downto 0)
);
end MUX;

architecture structural of MUX is

begin

Y(0)<= (A(0) AND Dec_Mux) OR (NOT (Dec_Mux) AND B(0));
Y(1)<= (A(1) AND Dec_Mux) OR (NOT (Dec_Mux) AND B(1));
Y(2)<= (A(2) AND Dec_Mux) OR (NOT (Dec_Mux) AND B(2));
Y(3)<= (A(3) AND Dec_Mux) OR (NOT (Dec_Mux) AND B(3));

end structural;
```

7.13 MUX8bit.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity mux8bit is
    port
    (
        --Input

        A : in std_logic_vector (7 downto 0);
```

```

        B : in std_logic_vector (7 downto 0);
        Dec_Mux : in std_logic;

--Output

        Y : out std_logic_vector (7 downto 0)

    );

end mux8bit;

architecture structural of mux8bit is
component MUX is
    port
    (
--Input

        A : in std_logic_vector (3 downto 0);
        B : in std_logic_vector (3 downto 0);
        Dec_Mux: in std_logic;

--Output

        Y : out std_logic_vector (3 downto 0)

    );

end component;

signal A_Signal1,A_Signal2,B_Signal1,B_Signal2,Y_Signal1,Y_Signal2 : std_logic_vector(3 downto 0)

begin

U1: MUX port map(A=>A(3 downto 0),B=>B(3 downto 0),Dec_Mux=>Dec_Mux,Y=>Y_Signal1);
U2: MUX port map(A=>A(7 downto 4),B=>B(7 downto 4),Dec_Mux=>Dec_Mux,Y=>Y_Signal2);
Y(3 downto 0)<= Y_Signal1;
Y(7 downto 4)<= Y_Signal2;

end structural;

```

7.14 decoderHex.vhd

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity decoderHex IS
PORT (      bin: in std_logic_vector(7 downto 0);
        clear : in std_logic;
        HEX0  : out std_logic_vector(7 downto 0);
        HEX1  : out std_logic_vector(7 downto 0);
        HEX2  : out std_logic_vector(7 downto 0)
);
END decoderHex;

Architecture logicFuntion of decoderHex is

component int7seg
PORT(      d      :IN STD_LOGIC_vector(3 downto 0);
        dOut      : out std_logic_vector(7 downto 0)
        );
END component;

component bin2bcd is
    Port ( bin : in  STD_LOGIC_VECTOR (7 downto 0);
          dec : out  STD_LOGIC_VECTOR (11 downto 0));
end component;

signal HEX0t: std_logic_vector(7 downto 0);
signal HEX1t: std_logic_vector(7 downto 0);
signal HEX2t: std_logic_vector(7 downto 0);
signal dec: std_logic_vector(11 downto 0);

BEGIN

HEX0 <= HEX0t when clear = '0' else "11111111";
HEX1 <= HEX1t when clear = '0' else "11111111";
HEX2 <= HEX2t when clear = '0' else "11111111";

U0: int7seg port map(dec(3 downto 0), HEX0t);
```

```

U1: int7seg port map(dec(7 downto 4), HEX1t);
U2: int7seg port map(dec(11 downto 8), HEX2t);

U3: bin2bcd port map (bin => bin, dec => dec);

END logicFuntion;

```

7.15 bin2dec.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity bin2bcd is
    port ( bin : in std_logic_vector (7 downto 0);
          dec : out std_logic_vector (11 downto 0));
end bin2bcd;

architecture Behavioral of bin2bcd is
    signal zeros : std_logic_vector (11 downto 0);

begin
    zeros <= (others => '0');

    b2b: process (bin, zeros)
        variable aux : std_logic_vector (19 downto 0);
        variable dig, ndig : std_logic_vector (3 downto 0);
    begin
        aux(19 downto 8) := zeros;
        aux(7 downto 0) := bin;

        for I in 0 to 2 loop
            aux(I+8 downto 1) := aux(I+7 downto 0);
        end loop; -- first 3 shifts do not need add3
    end process;
end architecture Behavioral of bin2bcd;

```

```

for I in 3 to 7 loop
  for J in 0 to 2 loop
    dig := aux(4*J+11 downto 4*J+8);
    case dig is
      when X"5" => ndig := X"8";
      when X"6" => ndig := X"9";
      when X"7" => ndig := X"A";
      when X"8" => ndig := X"B";
      when X"9" => ndig := X"C";
      when others => ndig := dig;
    end case;
    aux(4*J+11 downto 4*J+8) := ndig;
  end loop;  -- J
  aux(19 downto 1) := aux(18 downto 0);
end loop;  -- I
dec <= aux(19 downto 8);

end process b2b;

end Behavioral;

```

7.16 TLAB3.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity TLAB3 is
  port
  (
    MCLK   : in std_logic;
    Rst    : in std_logic;
    Step   : in std_logic;

```

```

        Start : in std_logic;
        X      : in std_logic_vector(3 downto 0);
        Cy     : out std_logic;
        sum    : out std_logic_vector(7 downto 0);
        HEX0, HEX1, HEX2 : out std_logic_vector(7 downto 0)
    );
end TLAB3;

```

architecture structural of TLAB3 is

component ASM is

```

port(
    reset      : in std_logic;
    clk        : in std_logic;
    start      : in std_logic;
    step       : in std_logic;
    EN_Reg,RST_Reg,Dec_Mux,EN_ACC,RST_ACC: out std_logic
);
end component;

```

component reg1bit is

```

port(
    x : in std_logic;
    clk : in std_logic;
    RST : in std_logic;
    EN : in std_logic;
    xout : out std_logic
);
end component;

```

component datapath is

```

    port(
        xi: in std_logic_vector(3 downto 0);
        clk: in std_logic;
        EN_Reg,RST_Reg,Dec_Mux,EN_ACC,RST_ACC : in std_logic;

        R : out std_logic_vector(7 downto 0);
        CY : out std_logic
    );
end component;

```

```

    );

end component;

component decoderHex IS
PORT (
    bin: in std_logic_vector(7 downto 0);
    clear : in std_logic;
    HEX0 : out std_logic_vector(7 downto 0);
    HEX1 : out std_logic_vector(7 downto 0);
    HEX2 : out std_logic_vector(7 downto 0)
);
END component;

signal EN_Reg_S,RST_Reg_S,Dec_Mux_S,EN_ACC_S,RST_ACC_S,CY_S,CY_Y: std_logic;
signal R_S,HEX0_S,HEX1_S,HEX2_S : std_logic_vector(7 downto 0);
signal stepDE10 : std_logic;
begin
stepDE10<= not(Step);
U0 : ASM port map(reset=>Rst,clk=>MCLK,start=>Start,step=>stepDE10,EN_Reg=>EN_Reg_S,RST_Reg=>RST_Reg_S,Dec_Mux=>Dec_Mux_S,EN_ACC=>EN_ACC_S,RST_ACC=>RST_ACC_S,CY=>CY_S,CY_Y=>CY_Y);
U1 : datapath port map(xi=>X,clk=>MCLK,EN_Reg=>EN_Reg_S,RST_Reg=>RST_Reg_S,Dec_Mux=>Dec_Mux_S,EN_ACC=>EN_ACC_S,RST_ACC=>RST_ACC_S,CY=>CY_S,CY_Y=>CY_Y);
U2 : decoderHex port map(bin=>R_S,clear=>'0',HEX0=>HEX0_S,HEX1=>HEX1_S,HEX2=>HEX2_S);
U3 : reg1bit port map(x=>CY_S, clk => MCLK, RST=> RST_ACC_S, EN=>CY_S, xout=>CY_Y);
HEX0<=HEX0_S;
HEX1<=HEX1_S;
HEX2<=HEX2_S;
Sum<=R_S;
Cy<= CY_Y;

end structural;

```

8 Pinos

ENTRADAS	
X(0)	C10
X(1)	C11
X(2)	D12
X(3)	C12
MCLK	P11
start	A12
rst	A13
step	B8

Tabela 1: Atribuição de pinos para as entradas.

SAÍDAS										
Sinal	Pino		HEX0	Pino		HEX1	Pino		HEX2	Pino
Sum(0)	A8		HEX0[0]	C14		HEX1[0]	C18		HEX2[0]	B20
Sum(1)	A9		HEX0[1]	E15		HEX1[1]	D18		HEX2[1]	A20
Sum(2)	A10		HEX0[2]	C15		HEX1[2]	E18		HEX2[2]	B19
Sum(3)	B10		HEX0[3]	C16		HEX1[3]	B16		HEX2[3]	A21
Sum(4)	D13		HEX0[4]	E16		HEX1[4]	A17		HEX2[4]	B21
Sum(5)	C13		HEX0[5]	D17		HEX1[5]	A18		HEX2[5]	C22
Sum(6)	E14		HEX0[6]	C17		HEX1[6]	B17		HEX2[6]	B22
sum(7)	D14		HEX0[7]	D15		HEX1[7]	A16		HEX2[7]	A19
Cy	A11									

Tabela 2: Atribuição de pinos para as saídas.